



UNIVERSITY OF CAMBRIDGE

Eigen-value Calculation for Sparse Matrices

Gabriel Nilsson

Cavendish Laboratory, Department of Physics, J J Thomson Avenue, Cambridge. CB3 0HE

Abstract

This report presents the implementation and analysis of the power iteration algorithm with Wielandt deflation for computing eigenvalues and eigenvectors of sparse matrices stored in Compressed Row Storage format. The algorithm is applied to matrices arising from finite volume discretisation of elliptic (Poisson) and hyperbolic-origin (Helmholtz) partial differential equations in one and two spatial dimensions, as well as random symmetric positive-definite matrices. Analytical eigenvalue expressions are derived for each matrix family and used to validate the numerical results. The convergence rate is shown to be governed by the eigenvalue ratio $r = |\lambda_2/\lambda_1|$, with empirical measurements matching theoretical prediction to high precision. A key finding is that the iteration count scales as $O(n^2)$ for one-dimensional discretisations but only $O(n)$ for two-dimensional discretisations, a difference traced to the dependence of the spectral gap on the underlying grid dimension rather than the matrix dimension. The stability of the deflation process is investigated, revealing a failure mode arising from the interaction of eigenvalue degeneracy, deflation ordering, and sub-unit eigenvalue magnitude. OpenMP parallelisation of the sparse matrix-vector product is implemented and benchmarked, achieving moderate speedup limited by the memory-bandwidth-bound nature of the operation. The limitations of the algorithm for non-symmetric matrices are discussed, and extensions including re-orthogonalisation and shift-and-invert strategies are proposed.

Abridged version. This is a shortened version of a 21-page report submitted for the MPhil in Scientific Computing at the University of Cambridge. Headings marked *[omitted]* show where sections have been removed. Nothing in the remaining text has been reworded. The full report is available on request.

1. Introduction and objectives *[omitted]*

2. Power Iteration and Matrix Deflation Algorithms

2.1. The Power Iteration Method

The power iteration method (also known as the Von Mises iteration) is the simplest iterative algorithm for computing the dominant eigenvalue and its associated eigenvector of a matrix. Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the method uses the fact that repeated multiplication of an arbitrary vector by $\mathbf{A}$ causes the component aligned with the dominant eigenvector to grow fastest, eventually dominating all other components. A core advantage of this method is that it does not require n-by-n

storage, as long as we can perform matrix-vector multiplications, making it useful for large sparse matrices. [1]

2.1.1. Theoretical Foundation *[omitted]*

2.1.2. Power Iteration Algorithm

The practical implementation uses normalisation at each iteration to avoid numerical overflow or underflow.

2.1.3. Convergence Rate

The rate of convergence is governed by the ratio of the two largest eigenvalues in magnitude. At iteration k , the error in the eigenvector direction satisfies:

$$\|\mathbf{u}_k - \mathbf{v}_1\| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right). \quad (1)$$

The convergence is therefore *linear* (first-order), with convergence factor $r = |\lambda_2/\lambda_1|$. [1, 2]

Algorithm 1 Power Iteration for the Dominant Eigenvalue

Require: Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$; tolerance $\varepsilon > 0$; maximum iterations $k_{\max}$

Ensure: Dominant eigenvalue λ_1 ; corresponding eigenvector $\mathbf{v}$

```

1: Generate random initial vector  $\mathbf{u}_0 \in \mathbb{R}^n$ 
2:  $\mathbf{u}_0 \leftarrow \mathbf{u}_0 / \|\mathbf{u}_0\|_2$   $\triangleright$  Normalise to unit magnitude
3: for  $k = 1, 2, \dots, k_{\max}$  do
4:    $\mathbf{w} \leftarrow \mathbf{A} \mathbf{u}_{k-1}$   $\triangleright$  Matrix-vector product
5:    $\mathbf{u}_k \leftarrow \mathbf{w} / \|\mathbf{w}\|_2$   $\triangleright$  Re-normalise
6:   if  $\|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2 < \varepsilon$  or  $\|\mathbf{u}_k + \mathbf{u}_{k-1}\|_2 < \varepsilon$  then  $\triangleright$ 
     Check convergence (sign may flip)
7:     break
8:   end if
9: end for
10:  $\mathbf{w} \leftarrow \mathbf{A} \mathbf{u}_k$ 
11:  $\lambda_1 \leftarrow \mathbf{u}_k^T \mathbf{w}$   $\triangleright$  Rayleigh quotient
12: return  $\lambda_1, \mathbf{u}_k$ 

```

The number of iterations required to achieve a tolerance ε can be estimated as:

$$k \approx \frac{\ln(\varepsilon)}{\ln |\lambda_2/\lambda_1|} \quad (2)$$

2.1.4. Storage Requirements [omitted]

2.1.5. Computational Cost per Iteration

Each iteration of the power method is dominated by one sparse matrix-vector product. For a matrix with N_{nz} non-zero entries, this costs $O(N_{\text{nz}})$ floating-point operations (FLOPs). The normalisation and convergence check each cost $O(n)$. Given sufficient sparsity, $O(N_{\text{nz}}) \approx cO(n)$ for some constant c , and the total FLOP per iteration scale as:

$$C_{\text{iter}} = O(N_{\text{nz}} + 2n) \approx O(N_{\text{nz}}). \quad (3)$$

2.2. Matrix Deflation

The power iteration method only computes the dominant eigenvalue of the matrix. To obtain further eigenvalues, we use *deflation* to construct a modified matrix where the already-computed eigenvalue is removed, while keeping subsequent eigenvalues intact. Allowing us to apply the power iteration method again on this new matrix.

2.2.1. Wielandt Deflation (Rank-One Deflation)

After computing the dominant eigenpair of $\mathbf{A}$, $(\lambda_1, \mathbf{v}_1)$ where $\|\mathbf{v}_1\|_2 = 1$, the deflated matrix $\mathbf{B}$ is:

$$\mathbf{B}_1 = \mathbf{A} - \lambda_1 \mathbf{v}_1 \mathbf{v}_1^T, \quad (4)$$

and in general, for multiple eigenvalues:

$$\mathbf{B}_k = \mathbf{A} - \sum_j^k \lambda_j \mathbf{v}_j \mathbf{v}_j^T. \quad (5)$$

Algorithm 2 Eigenvalue Computation via Power Iteration with Matrix Deflation

Require: Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$; number of eigenvalues m ; tolerance ε

Ensure: Eigenvalues $\lambda_1, \dots, \lambda_m$ and eigenvectors

```

 $\mathbf{v}_1, \dots, \mathbf{v}_m$ 
1:  $\mathbf{B}_0 \leftarrow \mathbf{A}$ 
2: for  $j = 1, 2, \dots, m$  do
3:    $(\lambda_j, \mathbf{v}_j) \leftarrow \text{POWERITERATION}(\mathbf{B}_{j-1}, \varepsilon)$   $\triangleright$ 
     Algorithm 1
4:    $\mathbf{B}_j \leftarrow \mathbf{B}_{j-1} - \lambda_j \mathbf{v}_j \mathbf{v}_j^T$   $\triangleright$  Rank-one deflation
5: end for
6: return  $\{(\lambda_j, \mathbf{v}_j)\}_{j=1}^m$ 

```

2.2.2. Implementation Considerations for Sparse Matrices

It is important to note that the deflated matrix $\mathbf{B}_k$ in Equation (5) is not sparse, even when $\mathbf{A}$ is sparse, because the outer product $\mathbf{v}_j \mathbf{v}_j^T$ is a dense $n \times n$ rank-one matrix. Explicitly forming $\mathbf{B}_k$ would therefore destroy the sparsity structure and require $O(n^2)$ storage, negating the advantage of using a sparse format.

Instead, the matrix-vector product with $\mathbf{B}_k$ should be computed implicitly:

$$\mathbf{B}_k \mathbf{x} = \mathbf{A} \mathbf{x} - \sum_{j=1}^k \lambda_j \mathbf{v}_j (\mathbf{v}_j^T \mathbf{x}). \quad (6)$$

This requires one sparse matrix-vector product $\mathbf{A} \mathbf{x}$, costing $O(N_{\text{nz}})$; and for each previously computed eigenpair, one dot product and one scaled vector subtraction, both costing $O(n)$.

The total cost per iteration when computing the $(k+1)$ th eigenvalue is therefore:

$$C_{\text{iter}}^{(k+1)} = O(N_{\text{nz}} + kn). \quad (7)$$

2.2.3. Storage Requirements for the Deflation Process [omitted]

3. Results Part 1

3.1. Analytical expectations [omitted]

3.2. Eigen-spectrum

The eigen-spectra (Figures 1, 2) show a strong alignment between the power iteration algorithm and the theoretical expectation. The error plots (Figures 3, 4) show

that the error is roughly within two orders of magnitude from the double precision machine epsilon. With the noteworthy exception of the single outlier for the 2D Helmholtz grid (Figure 4), which is discussed further in Section 4.2.2.

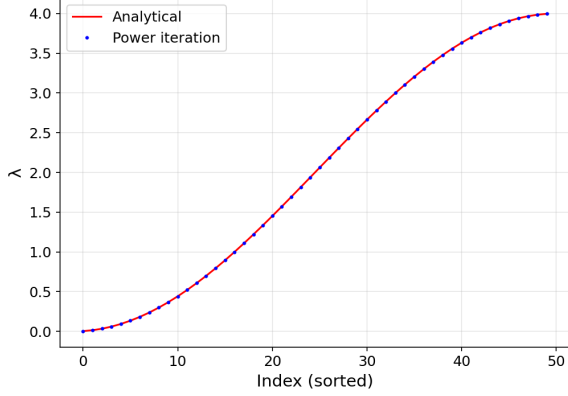


Figure 1: Eigenvalues in ascending order for tridiagonal 2nd-order 1D Laplacian matrix (Figure B.13), corresponding with a 1D Poisson PDE. Comparing result from power iteration algorithm with analytical expectation, at $n = 50$.

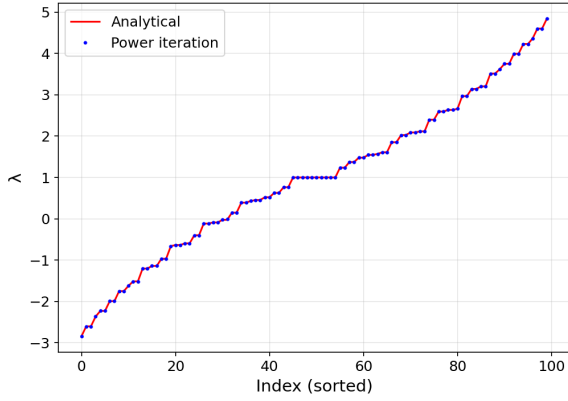


Figure 2: Eigenvalues in ascending order for 2nd-order 2D Helmholtz matrix (Figure B.14), corresponding with a 2D wave equation. Comparing result from power iteration algorithm with analytical expectation at $n = 100$.

4. Results Part 2: Solver Performance

This section evaluates the performance of the power iteration algorithm with Wielandt deflation (Algorithms 1-2) in terms of convergence rate, algorithmic stability, and execution time. Six matrix families are

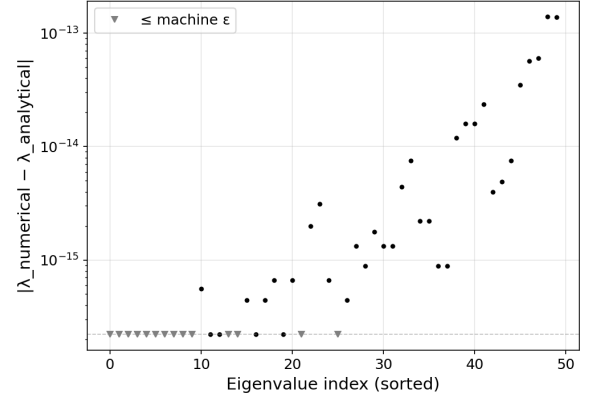


Figure 3: The absolute error between the numerical result and the analytical expectation between each eigenvalue, for the 2nd-order 1D Laplacian matrix (Figure B.13). Eigenvalues are sorted according to magnitude. Triangles represent errors smaller than the machine epsilon for double precision ($\epsilon = 2.22 \times 10^{-16}$).

considered: the 1D tridiagonal Laplacian, 1D pentadiagonal Laplacian, symmetric positive-definite matrices with random off-diagonal entries, 1D tridiagonal Helmholtz, 2D five-point Laplacian, and 2D five-point Helmholtz. Each family is benchmarked at varying sizes n to show scaling behavior.

4.1. Convergence Rate

To empirically confirm the convergence rate $r = |\lambda_2/\lambda_1|$, as described in Section 2.1.3, the following steps are taken: the power-iteration algorithm is run on matrix $\mathbf{A}$, keeping track of the Rayleigh quotient residual at each iteration for the first eigenvector. A second iteration is run, after deflation, to acquire the two dominant eigenvalues λ_1 and λ_2 . The theoretical convergence rate of $|\lambda_2/\lambda_1|^k$ is then compared with the residual-evolution recorded.

In Figures 5 & 6 we see confirmation that the convergence rate is highly aligned with the theoretical expectation.

A least-squares fit is made on the second half of each case to get a numerical value for the convergence rate. The difference in theoretical and empirical rates for the six cases range from $0.67 - 3.3 \times 10^{-10}$.

The observed convergence rates are consistent with the theoretical bound [1] $O(|\lambda_2/\lambda_1|^k)$ established in Section 2.1.3. The measured iteration counts of 20 000–30 000 for the 1D families at $n=100$ confirm the $O(n^2)$ scaling predicted by the spectral gap analysis in Section 4.1.

The observed residual convergence beats the theoretical line (red in Figures 5 & 6) because it is only the

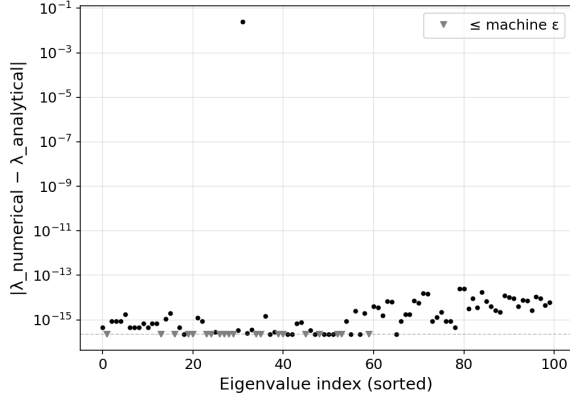


Figure 4: The absolute error between the numerical result and the analytical expectation between each eigenvalue, for the 2nd-order 2D Helmholtz matrix (Figure B.14). Eigenvalues are sorted according to magnitude. Triangles represent errors smaller than the machine epsilon for double precision ($\epsilon = 2.22 \times 10^{-16}$).

slowest decaying eigenvector (with the second largest eigenvalue) which decays according to $r = |\lambda_2/\lambda_1|$, all other eigenvectors will go to zero at a faster rate and will disappear simultaneously and early in the run, which is what is responsible for the initially high convergence rate.

In Figure 7 we see the convergence rate for each matrix type considered previously, as well as a symmetric positive-definite matrix with random off-diagonal entries (SPD). Revealing an order-of-magnitude difference in iteration count between the 1D families ($\sim 20\,000$ – $30\,000$) and the 2D families (~ 500 – 1000 iterations), excluding SPD. This difference is explained by how the spectral gap $\Delta\lambda = \lambda_1 - \lambda_2$ scales with matrix dimension in each case.

4.1.1. 1D Poisson ($r = 0.993$)

For the $(-1, 2, -1)$ tridiagonal matrix of dimension n , the two largest eigenvalues are $\lambda_n = 2 + 2\cos(\frac{\pi}{n+1})$ and $\lambda_{n-1} = 2 + 2\cos(\frac{2\pi}{n+1})$. Using the product-to-sum identity $\cos(\alpha) - \cos(\beta) = -2\sin(\frac{\alpha+\beta}{2})\sin(\frac{\alpha-\beta}{2})$, the spectral gap evaluates to:

$$\Delta\lambda = 4\sin(\frac{3\pi}{2(n+1)})\sin(\frac{\pi}{2(n+1)}) \approx \frac{3\pi^2}{(n+1)^2}, \quad (8)$$

for large n . Since $\lambda_{\max} \rightarrow 4$, the convergence ratio is:

$$r = \frac{\lambda_2}{\lambda_1} = 1 - \frac{\Delta\lambda}{\lambda_1}, \quad (9)$$

where $\lambda_1 = 2 + 2\cos(\frac{\pi}{n+1}) \rightarrow 4$ for large n ,

$$r \approx 1 - \frac{3\pi^2}{4(n+1)^2}. \quad (10)$$

At $n=100$ this gives $r \approx 0.9993$, matching the empirical value exactly. The required iteration count scales as $k \sim (n+1)^2$, i.e. quadratically in the matrix dimension $O(n^2)$.

4.1.2. 4th-order Laplacian ($r=0.9991$) [omitted]

4.1.3. 1d Helmholtz ($r=0.9988$) [omitted]

4.1.4. 2D Laplacian ($r=0.9698$)

The 2D five-point Laplacian in an $N \times N$ grid has matrix dimension $n = N^2$ and eigenvalues

$$\lambda_{k,l} = 4 - 2\cos(\frac{k\pi}{N+1}) - 2\cos(\frac{l\pi}{N+1}).$$

The dominant eigenvalue is $\lambda_{N,N} = 4 + 4\cos(\frac{\pi}{N+1})$, and the second largest is $\lambda_{N,N-1}$ (with multiplicity 2). The spectral gap is:

$$\begin{aligned} \Delta\lambda_{2D} &= 2\cos(\frac{\pi}{N+1}) - 2\cos(\frac{2\pi}{N+1}) \\ &\approx 2(1 - \frac{\pi^2}{2(N+1)^2}) - 2(1 - \frac{4\pi^2}{2(N+1)^2}) \\ &= \frac{3\pi^2}{(N+1)^2}, \end{aligned} \quad (11)$$

which is the same absolute gap as in the 1D case (Eq. 8), but now with $\lambda_{\max} \approx 8$ (versus 4 for 1D), and crucially with $N = \sqrt{n}$ rather than n . The ratio is therefore:

$$r_{2D} \approx 1 - \frac{3\pi^2}{8(N+1)^2} = 1 - \frac{3\pi^2}{8(\sqrt{n}+1)^2}. \quad (12)$$

At $N=10$ ($n=100$) this gives $r \approx 0.9698$, again matching the empirical value. The iteration count scales as $O(N^2) = O(n)$, i.e. linearly in the matrix dimension, not quadratically. This is the fundamental reason the 2D Laplacian (and the other 2D stencil matrices) converge much faster than the matrices stemming from 1D stencils.

4.1.5. 2D Helmholtz ($r=0.9511$) [omitted]

4.1.6. Symmetric positive-definite ($r=0.9829$) [omitted]

4.2. Algorithmic Stability

The power iteration algorithm with Wielandt deflation is numerically stable for the dominant eigenvalue, since a random starting vector is effectively guaranteed to have a non-zero component along the dominant eigenvector, and each iteration amplifies this component relative to all others. Stability issues arise, however, when deflation is applied repeatedly to recover the full spectrum.

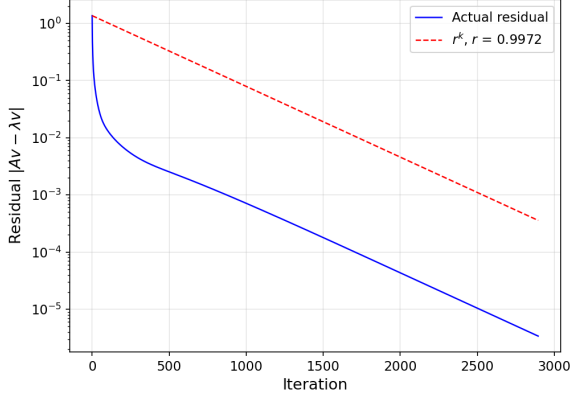


Figure 5: Empirical convergence rate compared with theoretical convergence rate for 2nd-order 1D Laplacian matrix (Figure B.13).

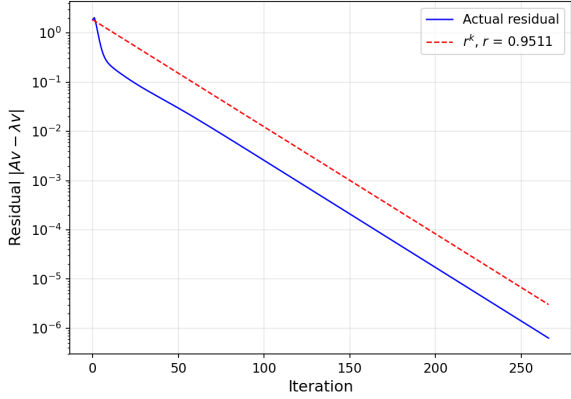


Figure 6: Empirical convergence rate compared with theoretical convergence rate for 2nd-order 2D Helmholtz matrix (Figure B.14).

4.2.1. Deflation noise accumulation

Each deflation step introduces numerical error through two mechanisms: (i) the computed eigenvector $\mathbf{v}_j$ is not exactly orthogonal to subsequent eigenvectors, so the rank-one subtraction $\lambda_j \mathbf{v}_j \mathbf{v}_j^T$ does not perfectly remove the j -th component; and (ii) the computed eigenvalue λ_j carries a small error, causing the subtraction magnitude to be slightly wrong. After k deflation steps, the deflated matrix $\mathbf{B}_k$ (Eq. 5) accumulates residual contributions from all k previously found eigenpairs.

For the structured matrices tested here, the measured pairwise orthogonality $|\mathbf{v}_i \cdot \mathbf{v}_j|$ remains below 10^{-8} for all $i \neq j$, even after 100 or 200 deflation steps. This level of orthogonality produces eigenvalue errors of $\approx 10^{-13}$ or below across the full spectrum, with one notable exception discussed below.

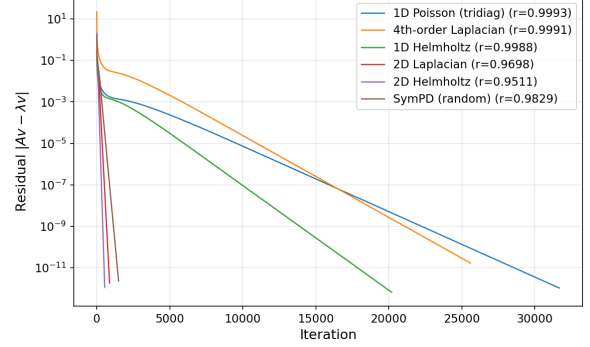


Figure 7: Residual $\|\mathbf{A}\mathbf{v} - \lambda\mathbf{v}\|_2$ versus iteration count for the dominant eigenvalue of each matrix family at $n=100$, averaged over 50 runs. The convergence ratio $r = |\lambda_2/\lambda_1|$ is shown in the legend.

4.2.2. Failure in 2D Helmholtz

The 2D Helmholtz matrix ($n=100$, $\kappa=3$) exhibits a single eigenvalue ($j=31$) reported as $\lambda=0$ instead of the true value $\lambda \approx -0.025$ (Figure 4).

After investigating residual size relative to noise from deflation, order in which eigenvalues are found, and the effect of deflation on degenerate eigenvalue pairs, it was found that this failure requires three simultaneous conditions:

- The eigenvalue is a member of a **degenerate** eigenvalue pair, i.e. there is a separate eigenvector which share the same eigenvalue.
- It is the **last eigenvalue** to be targeted by the power iteration algorithm.
- The eigenvalue is **less than 1**.

When power iteration with deflation finds the first member of a degenerate pair, the eigenvector $\mathbf{v}_a$ it produces becomes the starting vector for the next eigenvalue search. The deflated matrix $\mathbf{B}$ subtracts the contribution of $\mathbf{v}_a$, so the residual $\mathbf{B}\mathbf{v}_a$ reduces to $(\mathbf{A} - \lambda_a \mathbf{I})\delta\mathbf{v}$, where $\delta\mathbf{v}$ is the numerical error in $\mathbf{v}_a$. The component of this residual along the target eigenvector $\mathbf{v}_b$ is

$$(\lambda_b - \lambda_a)(\mathbf{v}_b \cdot \delta\mathbf{v}). \quad (13)$$

For a non-degenerate eigenvalue, this gap $\lambda_b - \lambda_a$ is nonzero, enabling well-behaved convergence. For a degenerate pair, $\lambda_b = \lambda_a$, making this factor exactly zero which removes the target component from the search vector, regardless of how large the eigenvector error $\delta\mathbf{v}$ is in the $\mathbf{v}_b$ direction. What remains after normalisation is a vector whose component along $\mathbf{v}_b$ is at machine-precision level ($\approx 10^{-17}$), indistinguishable from numerical noise.

This happens for every degenerate eigenvalue pair, which causes the power iteration algorithm to not find an eigenvector in the same degenerate set in consecutive runs, instead they are found in an interlaced order. If the most recent eigenvector came from degenerate set $\mathcal{A}$, eigenvectors in degenerate set $\mathcal{B}$ are not orthogonal from the seed and are found, causing the same but opposite problem after deflation, hiding subsequent eigenvectors from $\mathcal{B}$, making $\mathcal{A}$ visible again. Hence, the second problem with eigenvalue #31 is that it belongs to the degenerate set last explored, which is because its eigenvalue is the smallest of magnitude compared to all other eigenvalues. The deflation hides the eigenvectors degenerate to the one just found, which in this case is all remaining eigenvectors.

Even with an effectively orthogonal seed vector, recovery is still possible if $|\lambda_b| > 1$: each iteration of $\mathbf{B}$ multiplies the residual target component by λ_b , so even if the magnitude in the $\mathbf{v}_b$ direction is on the order of machine precision $\approx 10^{-17}$, it would grow exponentially and eventually rise above the accumulated deflation noise. However, for this degenerate pair $\lambda \approx -0.025$, making the seed instead shrink with every iteration and never emerge from the noise. The algorithm converges to the zero vector, reporting $\lambda = 0$ instead of the true eigenvalue, which is why we see the error on index 31 equal the actual eigenvalue that was supposed to be found, $|\lambda| \approx 0.025$ (Figure 4).

This diagnosis was confirmed empirically: the 1D Helmholtz at $n=200$ has a smaller final eigenvalue $|\lambda_{\min}| \approx 0.01$ and more deflation steps (200 vs. 100), yet it recovers even the final eigenvalue to machine precision because it is not a member of a degenerate pair.

Additionally, when we reset the seed vector to be a random seed vector right before pursuing the very last eigenpair, we fix the issue by effectively simulating the presence of an “interleaved” eigenvector to seed from, removing the failure condition number two. The result of which can be seen in Figure 8.

4.3. Execution time

4.3.1. Cost per iteration

Each power iteration step performs one sparse matrix-vector product ($O(\text{nnz})$ operations) followed by a normalisation ($O(n)$). The cost per iteration therefore scales linearly with the number of non-zero entries. For the tridiagonal matrix ($\text{nnz} = 3n - 2$) the measured cost per iteration grows from $0.4\mu\text{s}$ at $n = 10$ to $1.8\mu\text{s}$ at $n=100$. For the 2D Laplacian ($\text{nnz} = 5n - 4\sqrt{n}$) it grows from $3.6\mu\text{s}$ at $n=100$ to $55.9\mu\text{s}$ at $n=2500$, consistent with the linear-in- n scaling expected for a fixed-bandwidth sparse matrix.

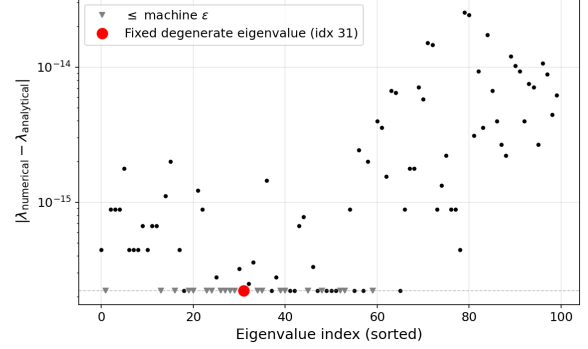


Figure 8: The absolute error between the numerical result and the analytical expectation between each eigenvalue, for the 2nd-order 2D Helmholtz matrix, after fixing the fault at eigenvalue #31. Eigenvalues are sorted according to magnitude. Triangles represent errors smaller than the machine epsilon for double precision ($\epsilon = 2.22 \times 10^{-16}$).

4.3.2. Total wall time [omitted]

4.3.3. Scaling with matrix dimension

The total wall time for computing the dominant eigenvalue is the product of the cost per iteration (Eq. 3) and the iteration count (Eq. 2). For a fixed-bandwidth sparse matrix with $N_{\text{nz}} = O(n)$, the cost per iterations is $O(n)$. The iteration count, however, depends on the convergence ratio $r(n)$, which differs fundamentally between 1D and 2D discretisations.

For the 1D tridiagonal Laplacian, the convergence ratio satisfies $r \approx 1 - \frac{3\pi^2}{4(n+1)^2}$ (Section 4.1). Using $\ln(1 - x) \approx -x$ for small x , the iteration count from Eq. 2 scales as

$$k_{1D} \approx \frac{|\ln(\epsilon)|}{\frac{3\pi^2}{4(n+1)^2}} = O(n^2). \quad (14)$$

The total wall time for the dominant eigenvalue therefore scales as $T_{1D} = O(n) \cdot O(n^2) = O(n^3)$.

For the 2D Laplacian on an $N \times N$ grid ($n = N^2$), the convergence ratio is $r \approx 1 - \frac{3\pi^2}{8(\sqrt{n}+1)^2}$ (Eq. 12), giving

$$k_{2D} \approx \frac{|\ln(\epsilon)|}{\frac{3\pi^2}{8(\sqrt{n}+1)^2}} = O(n). \quad (15)$$

Since the five-point stencil also has $N_{\text{nz}} = O(n)$, the total wall time is $T_{2D} = O(n) \cdot O(n) = O(n^2)$. The Helmholtz families follow the same scaling classes as their Laplacian counterparts, but with smaller prefactors due to the improved convergence ratio from the spectral shift.

Observed scaling and the logarithmic correction.

Figure 9 shows measured iteration counts, and Figure 10

the wall-time, versus matrix dimensions for all four matrix families on a log-log scale. Least-squares fits to the data yield effective exponents of approximately 1.8 for the 1D iteration count (versus the asymptotic prediction of 2) and 0.9 for the 2D iteration count (versus 1). The discrepancy can be explained by the convergence criterion.

The algorithm terminates when the eigenvector step norm $\|\mathbf{u}_{k+1} - \mathbf{u}_k\|$ falls below a tolerance ε . For normalised power iterates, consecutive iterates differ by $\mathbf{u}_{k+1} - \mathbf{u}_k = c(1 - r)r^k \mathbf{e}_2$, where $\mathbf{e}_2$ is the subdominant eigenvector direction and c depends on the initial vector. To leading order,

$$\|\mathbf{u}_{k+1} - \mathbf{u}_k\| \approx |c|(1 - r)r^k. \quad (16)$$

The factor $(1 - r)$ arises because the fractional change in the subdominant component is $(1 - r)$. Setting this equal to ε and solving for k :

$$k = \frac{\ln(\varepsilon) - \ln(1 - r)}{\ln(r)}. \quad (17)$$

Substituting the 1D expressions $1 - r \approx C_1/n^2$ and $\ln(r) \approx -(1 - r) \approx C_1/n^2$ (with $C_1 = 3\pi^2/4$), and noting that $\ln(1 - r) = \ln(C_1) - 2\ln(n)$, this becomes

$$k \approx \frac{n^2}{C_1} (|\ln(\varepsilon)| + \ln(C_1) - \ln(c) - 2\ln(n)). \quad (18)$$

The $-2\ln(n)$ term in the numerator reduces k below the pure $O(n^2)$ scaling of Eq. 14. Differentiation $\ln(k)$ with respect to $\ln(n)$ gives the effective log-log slope

$$\frac{d\ln(k)}{d\ln(n)} = 2 - \frac{2}{|\ln(\varepsilon)| + \ln(C_1) - \ln(c) - 2\ln(n)}, \quad (19)$$

which is strictly less than 2 for all finite n . For $\varepsilon = 10^{-8}$, $|c| \approx 1$, and $n = 10$ -500, this predicts an effective slope of ≈ 1.79 , in close agreement with the measured value of ≈ 1.79 . The same mechanism reduces the 2D effective slope from 1 to ≈ 0.9 , again aligning with what is observed in Figure 9.

The wall-time inherits this sub-asymptotic scaling from the iteration count (Figure 10). The measured time exponents (≈ 2.1 for 1D, ≈ 1.7 for 2D) are correspondingly below the asymptotic predictions of 3 and 2, consistent with the same logarithmic correction compounded with the per-iteration cost.

5. Discussion

The results of this work demonstrates that the power iteration method with Wielandt deflation is a viable approach for computing the eigenvalues of sparse matrices

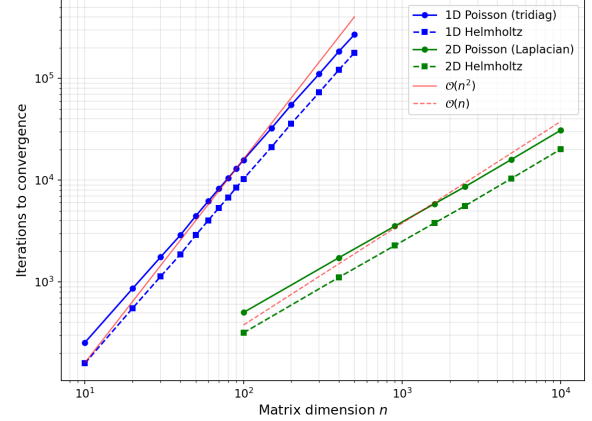


Figure 9: Iterations taken to converge (threshold of 10^{-8}) against the matrix dimension n for 1D systems and 2D systems. Compared with theoretically expected scaling order.

arising from PDE discretisation, but its practical performance is strongly governed by the spectral properties of the matrix at hand.

The convergence analysis in Section 4.3.3 revealed that for 1D discretisation matrices the iteration count scales as $O(n^2)$, whereas for 2D discretisation matrices it scales as $O(n)$, where n is the matrix dimension. This difference arises because the spectral gap near the dominant eigenvalue is determined by the underlying grid dimension N , not the matrix dimension. For 1D problems $N=n$, but for 2D problems $N=\sqrt{n}$, yielding a fundamentally better convergence ratio $r \approx 1 - O(\frac{1}{n})$ instead of $r \approx 1 - O(\frac{1}{n^2})$. The Helmholtz shift further improves convergence by reducing the dominant eigenvalue magnitude while preserving the absolute spectral gap, making relative difference between eigenvalues smaller.

The deflation process introduces two additional concerns: a linearly growing per-iteration cost $O(N_{nz} + kn)$ for the $(k+1)$ -th eigenvalue (Eq. 7), and accumulation of eigenvector error that contaminates subsequent deflation steps. The error plots confirm that later eigenvalues are computed with systematically lower accuracy, consistent with the loss of orthogonality inherent in Wielandt deflation without re-orthogonalisation.

5.1. Parallelisation of the algorithm

The power iteration algorithm has a straightforward structure that is amenable to parallelisation at several levels. OpenMP shared-memory parallelism was implemented for the sparse matrix-vector product, the dot product, and the vector norm operations, and its effectiveness was benchmarked on a 4-core machine.

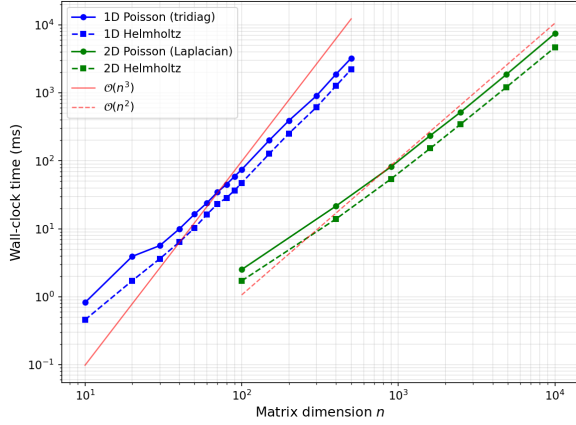


Figure 10: Wall time taken to converge (threshold of 10^{-8}) over the matrix dimension n for 1D systems and 2D systems. Compared with theoretically expected scaling order.

5.1.1. Parallelism within the sparse matrix-vector product [omitted]

5.1.2. Measured speedup

Figures 11 & 12 show the measured speedup of the sparse matrix-vector product on a 4-core machine for matrices ranging from $n = 50$ (148 non-zeros) to $n = 2500$ (12 300 non-zeros). For small matrices ($n \leq 100$), OpenMP provides no benefit and can even degrade performance: the thread fork/join overhead (typically 1-10 μ s) exceeds the serial computation time of the sparse matrix-vector product itself. For larger matrices, moderate speedup is achieved: the 2D Laplacian at $n = 2500$ reaches a speedup of $\approx 2.4\times$ on 4 threads, while a random matrix with $n=500$ and ≈ 10 non-zeros per row achieves $\approx 2.1\times$.

The speedup remains well below the ideal $4\times$ because the sparse matrix-vector product is memory-bandwidth-limited, because each non-zero element requires loading a matrix value, a column index, and a vector element, but performs only two floating-point operations (multiply and add). The arithmetic intensity is therefore $\approx 2/(8 + 4 + 8) = 0.1$ FLOPs/byte, placing the operation firmly in the bandwidth-bound regime of the roofline model. Increasing the thread count beyond 2 provides diminishing returns as the memory bus saturates.

For the matrix sizes typical of this project ($n = 50$ -2500), the per-iteration wall-clock time ranges from 1-37 μ s at 1 thread. The total eigenvalue computation time is dominated by the iteration count (which scales as $O(n^2)$ for 1D problems), not by the per-iteration cost. Consequently, OpenMP parallelism of the sparse matrix-vector product provides only a modest reduction

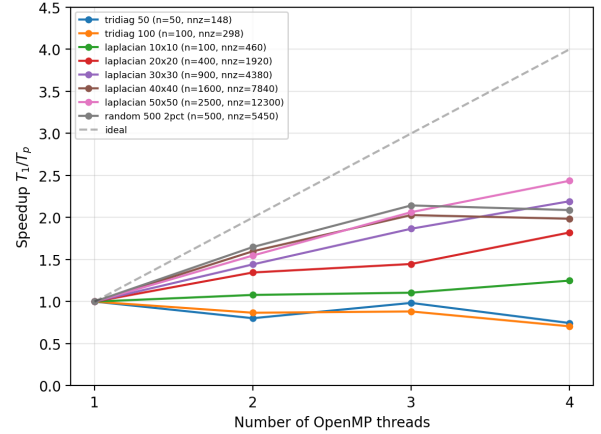


Figure 11: Speedup from OpenMP simple OpenMP parallelisation relative to serial implementation, over number of threads used. Matrix types and sizes are found in legend.

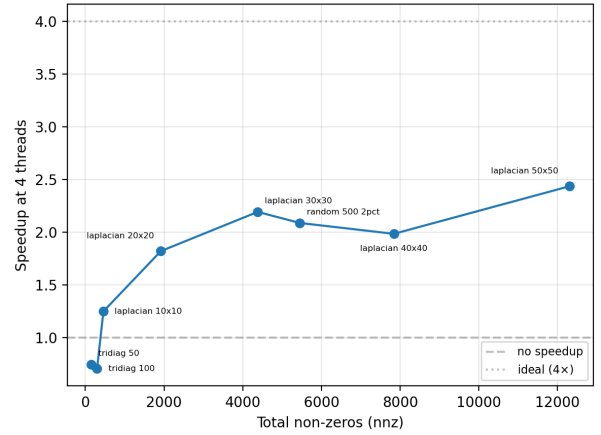


Figure 12: Speedup factor when using OpenMP implementation at 4 threads, for different matrix types, over the number of total non-zero values the matrix contains.

in total wall-clock time for these problem sizes.

5.1.3. Parallelism in the deflation projections [omitted]

5.1.4. Parallelism across eigenvalues [omitted]

5.2. Handling of symmetric and asymmetric matrices [omitted]

6. Conclusions

This project implemented the power iteration algorithm with Wielandt deflation for computing eigenvalues and eigenvectors of sparse matrices stored in CRS format. The algorithm was applied to six families of matrices arising from finite volume discretisation of elliptic (Poisson) and hyperbolic-origin (Helmholtz) PDEs, and a random symmetric positive-definite family.

It was found that the convergence rate is entirely determined by the eigenvalue ratio $r = |\lambda_2/\lambda_1|$, and the measured convergence ratios agree with theoretical predictions to four significant figures across all matrix families. The iteration count scaling depends on the spatial dimensionality of the underlying PDE: 1D discretisations yield $O(n^2)$ iterations while 2D discretisations yield $O(n)$ for matrix dimensions n . The Helmholtz spectral shift consistently improves convergence relative to the corresponding Laplacian.

The Wielandt deflation process enables computation of the full eigenspectrum using only $O(kn)$ additional storage for k eigenvalues, preserving the memory advantage of sparse storage. However, it introduces cumulative error in later eigenvalues due to loss of orthogonality, and its sequential nature limits parallelisation to within-iteration operations.

6.1. Further work

Several extensions would improve the practical applicability of the algorithm. For example, implementing a periodic Gram-Schmidt re-orthogonalisation of the computed eigenvectors, which would control the error accumulation observed in later eigenvalues at a cost of $O(k^2n)$ per re-orthogonalisation step.

Another extension would be shift-and-invert power iteration, in which $\mathbf{A}$ is replaced by $(\mathbf{A} - \sigma \mathbf{I})^{-1}$, transforming the eigenvalue nearest to a target σ into the dominant eigenvalue. This would enable computation of interior eigenvalues, however it would be at the expense of requiring a sparse direct or iterative linear solver.

On the parallelisation front, the current OpenMP implementation (Section 5.1) achieves only modest speedup due to the low arithmetic intensity of the sparse matrix-vector product. For larger matrices, parallelisation would yield stronger benefits.

Appendix A. Dirichlet Boundary Derivation *[omitted]*

Appendix B. Matrix structures

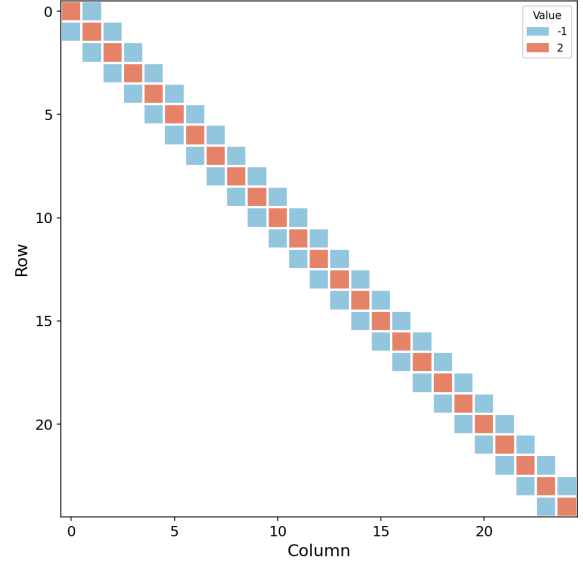


Figure B.13: Matrix structure for 2nd-order 1D Laplacian, with stencil $(-1, 2, -1)$, $n = 25$.

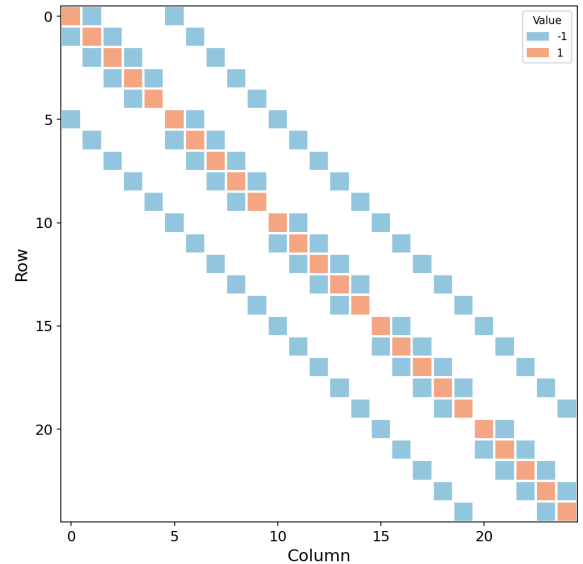


Figure B.14: Matrix structure for 2nd-order 2D Helmholtz for a 5x5 grid. $n = 25$.

Appendix C. Eigen-spectra *[omitted]*

Appendix D. Error plots *[omitted]*

Appendix E. Convergence plots *[omitted]*

References

- [1] G. H. Golub, C. F. Van Loan, Matrix Computations, 4th Edition, Johns Hopkins University Press, Baltimore, MD, 2013.
- [2] Y. Saad, Numerical Methods for Large Eigenvalue Problems, revised Edition, no. 66 in Classics in Applied Mathematics, Society for Industrial and Applied Mathematics, Philadelphia, 2011. doi:10.1137/1.9781611970739.